Sprint 7 Deliverable Report

1. Summary of Work Done

In Sprint 7, the team focused on validating the functionality and quality of the NoteVault application. Functional and non-functional testing were conducted, including user acceptance testing and heuristic evaluation. Several usability and localization issues were identified and resolved. Static code analysis using SonarQube confirmed high code quality.

2. Test Plan

The objective of this test plan is to verify that the NoteVault application functions correctly after code refactoring and meets both functional and non-functional requirements. The testing ensures system stability, usability, and performance before final delivery.

2.1 Resources

- Team members: Dinal Maha Vidanelage, Sandip Ranjit, Swostika Lama, Twe He Gam Aung
- Development tools: IntelliJ (IDE), Java, JavaFX, Maven, SonarQube, Junit, Jenkins, MariaDB
- Testing tools: Manual testing (UAT)

2.2 Test Tasks

- **Functional Testing**
    - Login / Registration
    - Notes CRUD
    - Tagging
    - Export PDF
    - Localization
    - Theme switching
- **Non-functional Testing**
    - Usability (Heuristic Evaluation)
    - Performance (basic checks)
    - Reliability (DB recovery)
    - Security (basic validation, password handling)

3. Functional Testing (Unit + Integration)

After completing unit and integration testing, the system was evaluated to verify that all core functionalities of NoteVault operate correctly after code refactoring and cleanup. During testing, few functional and usability issues were identified such visibility of selected notebook title in dashboard, and missing labels for input fields in signup and login screens.

3.1 Security Improvements

During development, static code analysis using SonarQube identified a design vulnerability related to the exposure of internal mutable collections within entity classes.

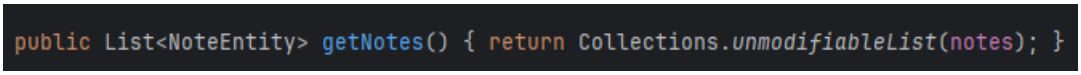
Issues identified:

- Direct exposure of internal collections (e.g., notes, tags) through getter methods
- Risk of unintended external modification of entity state
- Violation of encapsulation principlesThis significantly improved system security and reliability.

Solution implemented:

- Replaced direct collection returns with immutable views using Collections.unmodifiableSet()

Figure 1. Code snippet



Benefits:

- Prevents external code from modifying internal entity state
- Improves data integrity and consistency
- Strengthens encapsulation and object-oriented design
- Eliminates SonarQube security/code smell warnings

This improvement enhances the overall robustness and maintainability of the application.

3.2 UI Enhancement — Notebook Context Visibility

To improve usability, the dashboard was updated to:

- Display the selected notebook title
- Provide clearer context for notes organization

This change reduces confusion and improves navigation.

3.3 Testing Outcome

After implementing the above improvements:

- All core functional tests passed successfully
- No critical defects remained
- Data operations (backup/restore) worked reliably
- Security issues identified by SonarQube were resolved

4. UAT Results Table

Table 1 below shows the results of User Acceptance Testing (UAT) for the main features of the NoteVault application. The table is based on a set of functional test cases that cover important features such as login, note management, tagging, localization, theme settings, and PDF export. Each team member tested all cases using the final version of the system, and the results were recorded as pass or fail with short notes. All test cases were marked as “Pass,” which shows that the system works as expected. In the future, testing can be extended to include more edged cases and usability improvements.

Table 1. Table for User Acceptance Test results

Test Case	Type	Result	Notes
TC-1: User Login	Functional	Pass	Login works correctly
TC-2: User Registration	Functional	Pass	New user account stored in database
TC-3: Create Note & Notebook	Functional	Pass	Note saved in database
TC-4: Edit Note	Functional	Pass	Note updated in database
TC-5: Delete Note	Functional	Pass	Note deleted from database
TC-6: Add/Remove Tag Functionality	Functional	Pass	Tags displayed correctly
TC-7: Language Switch (Localization)	Functional	Pass	Selected language saved in system
TC-8: Theme Toggle (Light/Dark Mode)	Functional	Pass	Theme preference saved for the session
TC-9: Export Note as PDF	Functional	Pass	File successfully exported in PDF format
TC-10: Notebook management	Functional	Pass	Notebook changes are reflected in the database and UI

5. Bug Tracking Table

Table 2 presents the bugs identified during manual testing of the NoteVault application. The table includes details such as bug description, type, severity, status, and the implemented fixes. These issues were discovered while testing different parts of the system, especially the user interface and theme behavior. All reported bugs were fixed, improving the overall usability and consistency of the application. Future work can focus on preventing similar issues through additional testing and code reviews.

Table 2. Table for tracking bugs found during manual testing

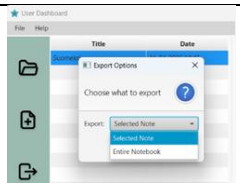
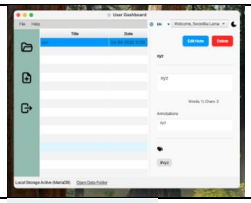
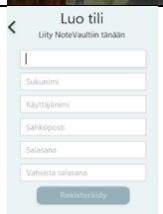
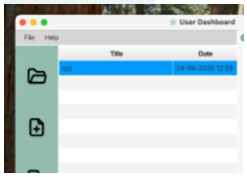
BUG ID	Description	Type	Severity	Status	Fix
BUG-01	Entry window labels not retained in Burmese language	UI	High	Fixed	Changed fonts to Noto Sans.
BUG-02	OK and Cancel Buttons in View and User Dashboard display previously selected language.	UI	High	Fixed	Create new *Ok* and *Cancel* buttons every time confirmation alert loads.
BUG-04	Manage User -> Update button Translation	UI	High	Fixed	Burmese translation was not provided for the word.
BUG-05	Rich text editor content color in dark mode (text was black in Create)	UI	High	Fixed	• Ensure theme is applied when the editor is initialized. • Force a default inline style range on the editor so newly typed characters inherit correct color.
BUG-06	Theme toggle is not persistent if toggled from Create dialog	UI	Low	Fixed	• Added a helper to apply theme to all open windows and use it from the toggle/set methods. • Toggle logic now updates every open Scene and publishes an event so controllers can react.
BUG-07	Toggle theme icons not updated	UI	Medium	Fixed	• Added EventBus notification when theme changes (see above). • Subscribed dashboard and create controllers to ThemeChangedEvent and update their icons when the event is received.
BUG-08	The delete confirmation dialog displayed the	UI	Medium	Fixed	Updated handleDelete() to fetch the note title directly from the table column.

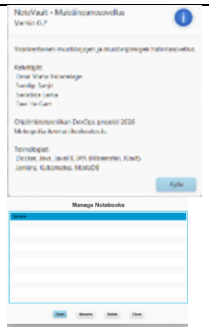
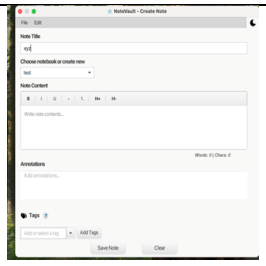
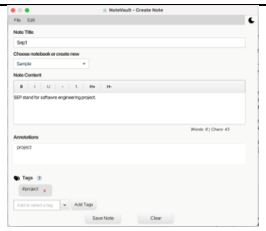
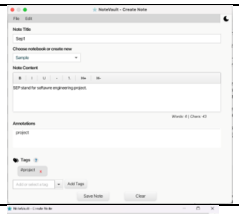
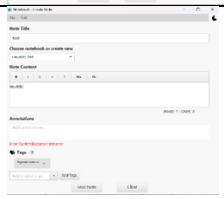
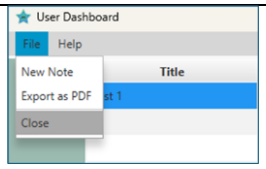
	window title "Delete Note" instead of the actual note name				

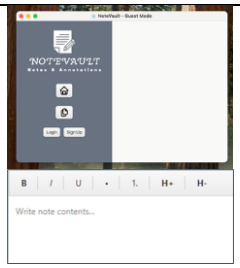
6. Heuristic Evaluation Report

Table 3 presents the results of the heuristic evaluation conducted on the NoteVault application. The evaluation was based on Nielsen's usability heuristics and was carried out to identify usability issues in the system interface. Each issue was reviewed, documented with a severity rating, and supported with suggested improvements. The findings mainly highlight problems related to consistency, feedback, error handling, and user guidance, especially in areas such as dialogs, localization, and navigation. Most issues were related to UI clarity rather than core functionality. These results were used to improve the overall usability of the system by refining interface behavior and enhancing user experience.

Table 3. Group summary table for heuristic evaluation

No	Heuristic	Description of the issue	Screenshot	Severity	Suggested Improvement
1	H1-1: Simple & natural dialog	The PDF Export choices display the same 'Selected Note' option for both the default selection and one of the options.		1	Remove repeated identical options.
2	H1-2: Speak the users' language	"Local Storage Active (MariaDB)" may confuse non-technical users.		1	Rename with: "Local storage enabled" or hide technical database info from normal users.
3	H1-3: Minimize users' memory load	The View Note screen does not display the name of the currently selected notebook, and the Signup and Login screens rely solely on placeholder texts inside input fields without persistent labels, leaving users without clear context when fields are filled.	 	2	Display notebook name above the note title. Add persistent, localized labels above or to the left of each input and bind them to i18n keys.

4	H1-4: Consistency	Button colors are not consistent and Some of the buttons did not display consistent message type. For example, in About screen, the button should have been OK or Close but instead was YES.		2	Make Delete button color red and make sure that the buttons display consistent message type.
5	H1-5: Feedback	After saving or editing a note, the window closes immediately and navigates to Notes Dashboard window. Also, after clicking “Save Note”, the UI does not show a confirmation message.		3	Show a brief success message with a visible success message.
6	H1-6: Clearly marked exits	There are no undo/redo functions for deleting or renaming notebooks. Closing window loses unsaved data in create note		3	Add an unsaved changes confirmation dialog with Save, Discard, and Cancel options when closing, and implement undo functionality for deleted notes and notebooks.
7	H1-7: Shortcuts	There are no keyboard shortcuts for creating notes or opening notes and notebooks.		1	Add standard accelerators (Ctrl+S for Save, Ctrl+N for New Note, Ctrl+O.
8	H1-8: Precise & constructive error messages	In Create Note window, there is a raw exception text like RuntimeException test error, which non-technical users cannot understand.		1	Replace raw exception display with helpful, localized messages that explain cause and next steps (e.g., "Failed to save note. ").
9	H1-9: Prevent errors	The Close button in Dashboard window immediately closes the application without any confirmation. It can lead to user dissatisfaction.		3	Add confirmation dialog to ask user if they want to close the app.

10	H1-10: Help and documentation	The home and create icons on the dashboard lack text labels, making their purpose unclear to new users. Additionally, tooltips are missing on the text formatting toolbar buttons.		2	Add tooltips to all formatting toolbar buttons as well as label icons for clear info, so users can quickly identify their function.

7. SonarQube Report

During SonarQube analysis integrated with the Jenkins pipeline, the project achieved high ratings in reliability, maintainability, and security. No major code smells, bugs, or vulnerabilities were detected, and all quality gates passed successfully. Below are the screenshots from SonarQube analysis and Jenkins pipeline overview.

Figure 2. Screenshot of New Code analyzation from SonarQube dashboard

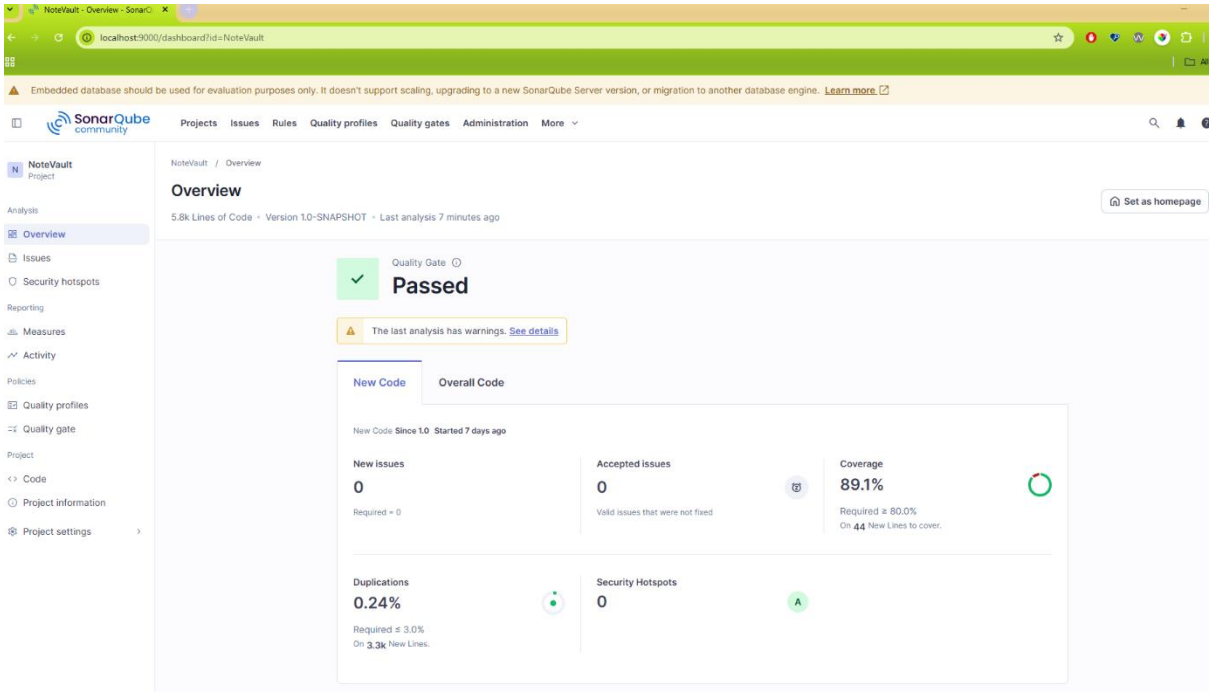


Figure 3. Screenshot of Overall Code Quality from SonarQube dashboard

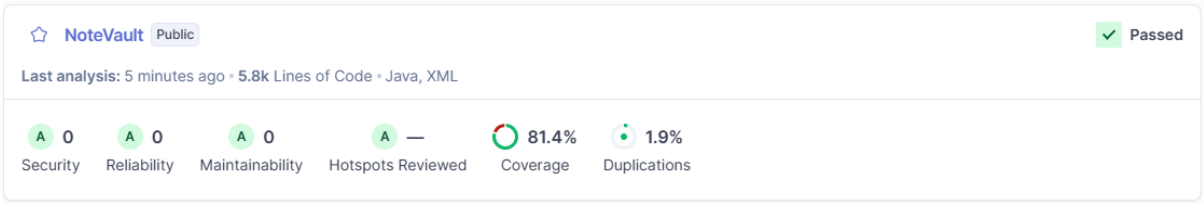


Figure 4. Metrics graph after SonarQube code quality analyzation

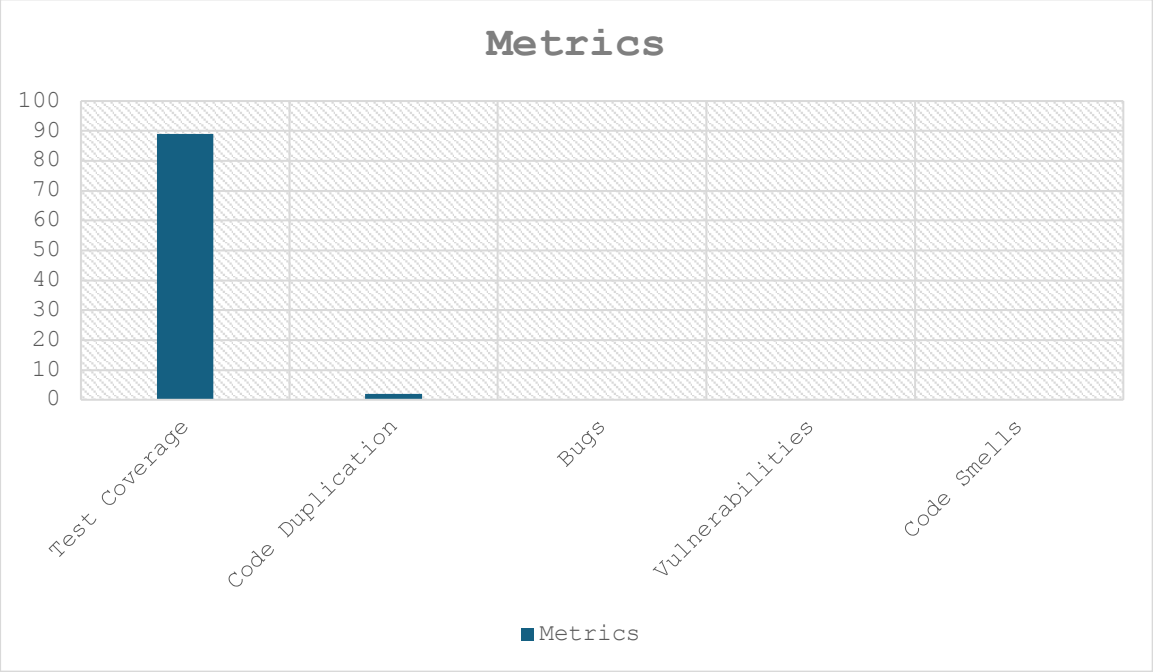
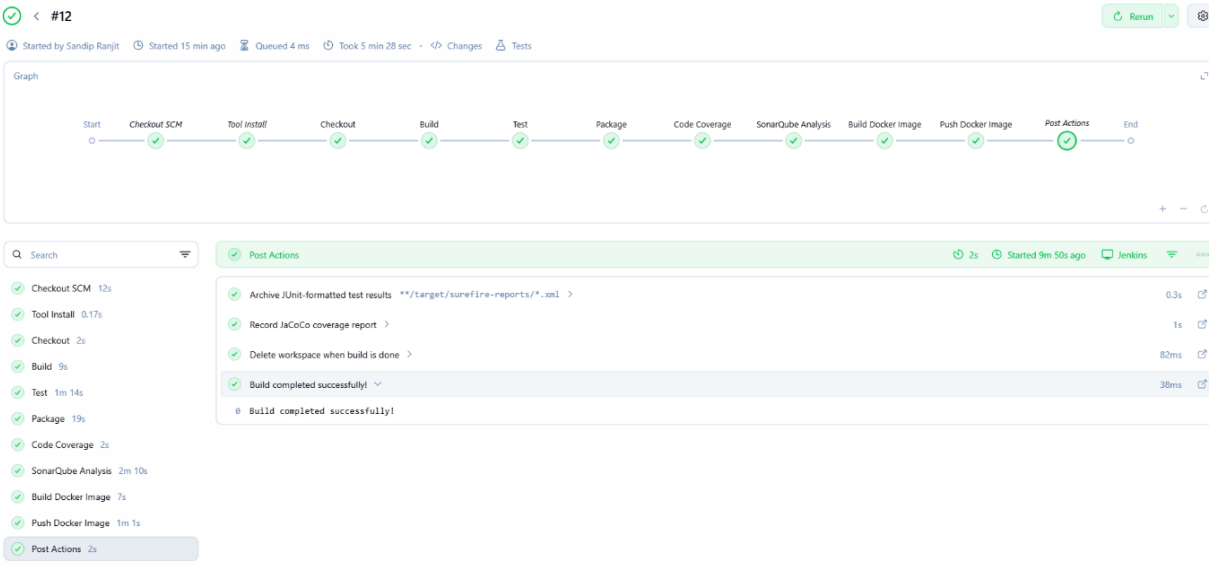


Figure 5. Screenshot of Jenkins pipeline



8. Technical Changes Based on Testing

8.1 Data Persistence & Backup Handling Improvements

Based on testing results, improvements were made to ensure reliable interaction between the application and the database.

- Improved separation between controller and service layers to ensure cleaner data access
- Ensured consistent loading of notebooks and notes through DashboardService
- Fixed issues related to UI not reflecting the currently selected notebook correctly
- Improved synchronization between UI state and persisted data

8.2 UI Consistency and Usability Fixes

Testing revealed multiple usability inconsistencies in UI components.

- Standardized button types across dialogs (e.g., OK/Cancel instead of YES/NO)
- Added missing input field labels in authentication screens to improve clarity
- Improved dashboard navigation by ensuring notebook context visibility

8.3 Error Handling and Feedback Improvements

During testing, some error messages and exception handling patterns were identified as non-user friendly.

- Replaced raw exception messages with user-friendly status messages
- Improved localization coverage for missing keys (e.g., login success messages)
- Standardized error handling using centralized utilities such as AlertUtil
- Improved logging for debugging while keeping UI messages clean

8.4 Security and Stability Enhancements

Static analysis using SonarQube identified design-level vulnerabilities in entity classes. Added ZIP validation checks (entry limits, path traversal prevention).

- Prevented exposure of internal mutable collections by returning immutable views
- Improved encapsulation and protected internal state from unintended modification

8.5 Code Quality Improvements

Testing feedback and SonarQube analysis led to refactoring efforts.

- Reduced duplicated logic across controllers
- Improved modularization of service layer
- Increased readability and maintainability of codebase
- Extended unit tests to cover additional edge cases and execution paths identified during testing

9. Documentation Updates

During Sprint 7 testing and implementation, several updates were made to system documentation and design-level understanding of the application.

9.1 System Architecture Updates

- Refined the service layer to better separate business logic from UI controllers
- Improved interaction between controller and service classes (e.g., DashboardService, PDFExportService)
- Strengthened overall application structure by enforcing clearer separation between UI, service, and utility layers
- Improved handling of asynchronous operations (e.g., background tasks for loading notes)

9.2 Specification Updates

- Updated UI specification to reflect:
    - Notebook title visibility in dashboard
    - Presence of tooltips and labels for better usability
- Extended localization specification to include missing translation keys discovered during testing (e.g., login.success)

9.3 Testing Strategy Updates

- Expanded unit testing scope to include additional branch-level test cases identified during debugging and SonarQube analysis.
- Included validation of edge cases such as empty states, missing translations, and concurrent operations
- Used static analysis tools (SonarQube) alongside testing to identify code quality and design issues

10 Performance Testing

Due to the absence of external load testing tools such as JMeter, a custom JUnit-based load testing framework was implemented to evaluate system performance under concurrent database operations.

The test simulates multiple users performing CRUD operations simultaneously on notes and notebooks. Test results are exported to CSV for further analysis and traceability.

10.1 Test Configuration

- Threads: 20 concurrent users
- Iterations per thread: 50 operations
- Total operations: 1000
- Operation types: Create, Read, Update, Delete (randomized distribution)

10.2 Metrics Collected

The system was evaluated using the following performance indicators:

- Average response time
- Minimum / Maximum latency
- 95th percentile latency (P95)
- Throughput (operations per second)
- Failure rate
- Execution time

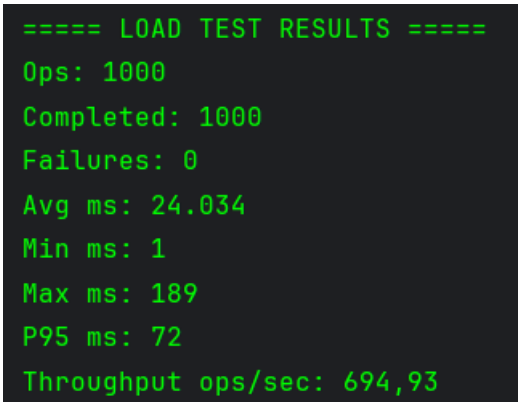


Figure 6. Screenshot of database load test result

10.4 Observations

- System handled concurrent database access without failures.
- No deadlocks or data corruption observed.
- Average latency remained low under load.
- Peak response time (210 ms) occurred under update-heavy operations but remained acceptable.
- Throughput indicates stable performance under simulated multi-user usage.

11 Contributions

Table 4. Table for recording of each group member's contributions

Member Name	Assigned Tasks	Time Spent (hrs)	In-class tasks
Dinal Maha Vidanelage	• Bug-fixing • Manual testing • Increase code coverage • Scrum Master	25	Submitted
Sandip Ranjit	• Bug-fixing • UAT tests • Increase SonarQube code coverage	30	Submitted
Swostika Lama	• Bug-identifying • Manual testing and documenting bugs • Wrote unit test to increase the sonar coverage.	27	Submitted
Twe He Gam Aung	• Bug-fixing • Refactor according to sonar standard and wrote unit test to cover more SonarQube code coverage. • Manual testing to find bugs	24	Submitted